
ANDREW THE TERRIBLE - SOLUCIÓN GUÍA

Universidad de Bogotá Jorge Tadeo Lozano

Alvarado Becerra Ludwig - Franco Calderón Alejandro
Estructuras de datos - 2026-1S

Índice

1. Interpretación del problema, entrada y salida de datos	1
1.1. Objetivo	1
1.2. Entrada de datos esperada	1
1.3. Salida de datos esperada	1
2. Solución conceptual	1
2.1. Obtener el índice del elemento máximo y mínimo de la lista	1
2.2. Cambiar los elementos de posición en la lista	2
2.2.1. Usando variable temporal	2
2.2.2. Operaciones aritméticas	2
2.2.3. Operaciones de XOR	3
3. Código modelo	4
3.1. Método para obtener el índice del elemento máximo y mínimo de una lista . .	4
3.2. Lectura de datos	5
3.3. Cambio de elementos de la lista	7
3.4. Salida de datos	7
3.5. Código final	8

1. Interpretación del problema, entrada y salida de datos

1.1. Objetivo

Modificar una lista cambiando la posición del elemento más grande de la lista con uno dado por Andrew o si inseguro, cambiarlo por la posición del elemento más pequeño de la lista.

1.2. Entrada de datos esperada

El enunciado nos dice de leer datos hasta que se le entregue un $n = 0$, por ejemplo, en el ejemplo del enunciado la primera línea es:

1

Este 1 es n , ahora se deben leer n departamentos, por eso mismo, tenemos que la segunda línea es:

Finance 5 0

De esta cadena podemos obtener; s , el nombre del departamento; m , el número de empleados en el departamento; I , el índice por el cual se debe cambiar de posición el elemento mayor de la lista. Teniendo en cuenta lo anterior, $s = \text{"Finance"}$, $m = 5$ y $I = 0$. Estos datos se pueden almacenar en variables con tipo de dato `str`, `int` y `int`, respectivamente.

Lo siguiente es leer m elementos que pueden ser guardados en una lista de enteros. Para el ejemplo que se está siguiendo, esta lista se va a llamar `reportes_financieros`:

[10, 25, 30, 5, 12]

Repetir esto con cada departamento

1.3. Salida de datos esperada

Teniendo leídos los datos, la solución que se debe imprimir en consola es el nombre del departamento y en la línea siguiente todos los elementos de la lista final (con los cambios en las posiciones) separados por espacio.

2. Solución conceptual

2.1. Obtener el índice del elemento máximo y mínimo de la lista

Tomando en cuenta la lista `reportes_financieros` que se mencionó en secciones anteriores. Se puede crear una función `get_max_index(lista)` cuya implementación siga estos pasos:

1. Crear dos variables `elemento_max` (inicializada con `reportes_financieros[0]`) y `indice_max` (inicializada en 0).
2. Iterar sobre los índices de la lista `reportes_financieros`. Esto se hace con una variable iteradora i , para el ejemplo de la lista [10, 25, 30, 5, 12] esta variable i va a tomar los valores de $i = \{0, 1, 2, 3, 4\}$.
3. Comparar `reportes_financieros[i]` con la variable `elemento_max`, si `reportes_financieros[i]` es mayor a `elemento_max`, entonces asigne

`reportes_financieros[i]` a `elemento_max` y el valor de la variable iteradora *i* a `indice_max`

4. Repetir hasta haber recorrido el final de la lista.
5. Retornar `indice_max`.

Para encontrar el índice del elemento mínimo de la lista se sigue un procedimiento similar para el índice máximo, únicamente se modifica el paso 3, utilizando variables con diferentes nombres, la función `get_min_index(lista)` deberá modificarse para incluir este paso:

- Comparar `reportes_financieros[i]` con la variable `elemento_min`, si `reportes_financieros[i]` es menor a `elemento_min`, entonces asigne `reportes_financieros[i]` a `elemento_min` y el valor de la variable iteradora *i* a `indice_min`.

Tener en cuenta hacer el cambio de nombre de las variables.

2.2. Cambiar los elementos de posición en la lista

Hay 3 posibles maneras de hacer un cambio de variable. Usar la que más sea fácil de entender para el programador.

2.2.1. Usando variable temporal

Tener en cuenta la lista que se ha venido manejando:

[10, 25, 30, 5, 12]

Se debe cambiar de posición el elemento 10 (posición 0) con 30 (posición 2), esto se puede lograr utilizando una variable temporal `temp` que se utiliza como ayuda para guardar la información de uno de los elementos de la lista.

1. Asignar `reportes_financieros[2]` a `temp`
2. Asignar `reportes_financieros[0]` a `reportes_financieros[2]`
3. Asignar `temp` a `reportes_financieros[0]`

De esta manera se logra hacer el cambio de variables, `temp` queda eliminada con el *garbage collector* de Python.

2.2.2. Operaciones aritméticas

Este funcionamiento se puede demostrar de manera general con un ejemplo, suponer que tenemos dos variables $a = x_1$ y $b = x_2$, el objetivo es que *a* tenga el valor de x_2 y *b* a x_1 , el primer paso es sumar el valor de *b* a *a*, esto resulta en:

$$a = x_1 + x_2$$

Ahora, el valor actual de *a* se debe restar con el valor actual de *b*:

$$b = \overbrace{(x_1 + x_2)}^a - x_2$$

Se puede cancelar ambos x_2 de la ecuación:

$$b = x_1 + \cancel{x_2} - \cancel{x_2}$$

Quedando finalmente $b = x_1$. Ahora únicamente falta restar el valor actual de b con a :

$$a = x_1 + x_2 - x_1$$

Cancelando ambos términos opuestos x_1 :

$$a = \cancel{x_1} + x_2 - \cancel{x_1}$$

Finalmente, queda $a = x_2$, cumpliendo con el objetivo del cambio de variable.

Los pasos a seguir algorítmicamente:

1. Asigne `reportes_financieros[0] + reportes_financieros[2]` a `reportes_financieros[0]`
2. Asigne `reportes_financieros[0] - reportes_financieros[2]` a `reportes_financieros[2]`
3. Asigne `reportes_financieros[0] - reportes_financieros[2]` a `reportes_financieros[0]`

2.2.3. Operaciones de XOR

Una operación XOR ($\oplus$) es una compuerta lógica que da verdadero cuando solo una de las entradas es verdadero.

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Para hacer un cambio de variable podemos inicializar dos variables $a = x_1$ y $b = x_2$ y hacer a XOR b :

$$a = a \oplus b = x_1 \oplus x_2$$

Ahora se vuelve a realizar esta misma operación de a XOR b pero ahora asignando a la variable b :

$$b = a \oplus b = (x_1 \oplus x_2) \oplus x_2$$

Se va a utilizar tres posibles propiedades de la operación XOR:

- $x \oplus x = 0$
- $x \oplus 0 = x$

- Asociativa y conmutativa.

Por lo tanto, b se convierte en:

$$b = x_1 \oplus (x_2 \oplus x_2) = x_1 \oplus 0 = x_1$$

Se hace la misma operación con la variable a :

$$a = a \oplus b = (x_1 \oplus x_2) \oplus x_1$$

Y se aplican las mismas propiedades:

$$a = (x_1 \oplus x_1) \oplus x_2 = 0 \oplus x_2 = x_2$$

Y así a termina con el valor original de b y viceversa.

Algorítmicamente los pasos a seguir en el problema es hacer esta misma operación XOR como se explicó anteriormente, en la mayoría de lenguajes de programación la operación XOR se hace con el operador $\wedge$, los pasos a seguir son:

1. Asignar `reportes_financieros[0] ^ reportes_financieros[2]` a `reportes_financieros[0]`
2. Asignar `reportes_financieros[0] ^ reportes_financieros[2]` a `reportes_financieros[2]`
3. Asignar `reportes_financieros[0] ^ reportes_financieros[2]` a `reportes_financieros[0]`

3. Código modelo

Se va a explicar parte a parte una posible solución al problema, recuerde que puede probar con diferentes implementaciones o incluso formulando más soluciones al problema, se invita al estudiante que haga ese ejercicio.

3.1. Método para obtener el índice del elemento máximo y mínimo de una lista

En primer lugar se debe definir qué es lo que se debe ingresar al método y qué debe retornar, es decir, recibir una lista de enteros y retornar un entero. Esto en Python se declara de la siguiente manera:

```
1 def get_max_index(lista: list[int]) -> int:
```

Luego, se debe inicializar las dos variables de `elemento_max` y `indice_max` que se mencionó anteriormente:

```
1 def get_max_index(lista: list[int]) -> int:
2     elemento_max = lista[0]
3     indice_max = 0
```

Ahora, se debe comenzar la iteración sobre los índices de la lista:

```

1 def get_max_index(lista: list[int]) -> int:
2     elemento_max = lista[0]
3     indice_max = 0
4     for i in range(len(lista)):

```

Y se sigue la lógica que se mencionó en la sección 2.1:

```

1 def get_max_index(lista: list[int]) -> int:
2     elemento_max = lista[0]
3     indice_max = 0
4     for i in range(len(lista)):
5         if (lista[i] > elemento_max):
6             elemento_max = lista[i]
7             indice_max = i

```

Finalmente, retornar `indice_max`:

```

1 def get_max_index(lista: list[int]) -> int:
2     elemento_max = lista[0]
3     indice_max = 0
4     for i in range(len(lista)):
5         if (lista[i] > elemento_max):
6             elemento_max = lista[i]
7             indice_max = i
8     return indice_max

```

Ahora, se implementa una estructura similar para el método de obtener el índice del elemento mínimo de la lista:

```

1 def get_max_index(lista: list[int]) -> int:
2     elemento_min = lista[0]
3     indice_min = 0
4     for i in range(len(lista)):

```

Lo único es cambiar la condición de acuerdo a la lógica previamente mencionada:

```

1 def get_max_index(lista: list[int]) -> int:
2     elemento_min = lista[0]
3     indice_min = 0
4     for i in range(len(lista)):
5         if (lista[i] < elemento_min):
6             elemento_min = lista[i]
7             indice_min = i
8     return indice_min

```

3.2. Lectura de datos

Una vez implementadas los métodos de ayuda, el enunciado establece que se nos da un primer dato n que establece el número de departamentos a leer, si este número es igual a 0

entonces se deja de leer datos:

```
1 n = int(input())
2
3 while (n != 0):
```

Ahora, se deben leer los n departamentos, se utilizar un ciclo `for` con rango hasta n , se utiliza una variable iteradora de `_` que generalmente se utiliza para nombre de una variable que no se va a utilizar dentro del código, únicamente se utiliza el ciclo para leer los n departamentos

```
1 for _ in range(n):
```

Ahora, toca leer una cadena con el nombre del departamento, la cantidad de empleados y el índice para cambiar la posición de la lista, esto se logra:

```
1 data = input().split()
```

La línea de `data = input().split()` crea una lista con la cadena que ingresan los datos, por ejemplo, si se ingresa la siguiente cadena:

"Finance 5 0"

Después de esa línea, la variable `data` va a ser igual a:

```
1 print(data)
2 ["Finance", "5", "0"]
```

Ahora, se organizan los datos en tres variables; una cadena y dos enteros.

```
1 department = data[0]
2 employees, I = int(data[1]), int(data[2])
```

Se debe de leer los reportes financieros, estos vienen ingresados en una cadena, se deben transformar a una lista con ayuda de la función `list` y `map`, se recomienda leer la documentación de python para la función `map`

```
1 financial_reports = list(map(int, input().split()))
```

Por último, por fuera del ciclo `for` se vuelve a pedir n

```
1 n = int(input())
```

Esta parte completa de lectura de datos se ve conjunta como:

```
1 n = int(input())
2
3 while (n != 0):
4
5     for _ in range(n):
6         data = input().split()
7         department = data[0]
8         employees, I = int(data[1]), int(data[2])
9         financial_reports = list(map(int, input().split()))
```



```
10
11     n = int(input())
```

3.3. Cambio de elementos de la lista

Lo primero es obtener el índice del elemento máximo de la lista, es decir, dentro del contexto del problema es el amigo de Andrew, para esto se llama a la función `get_max_index()`

```
1 I_max = get_max_index(financial_reports)
```

En el problema se comenta que Andrew nos dará un índice para cambiar el elemento con el del máximo de la lista, este índice ya se tiene guardado en la variable `I`, si es diferente a `-1`, entonces podemos hacer el cambio con el índice que dice Andrew:

```
1 if (I != -1):
2     financial_reports[I], financial_reports[I_max] =
        financial_reports[I_max], financial_reports[I]
```

Se hace entonces el cambio de elemento utilizando la lógica de XOR pero en forma *pythonesca*, sin embargo, se puede hacer sin problema de las diferentes formas que se explicaron en la sección 2.2.

En caso de que Andrew esté inseguro de qué elemento cambiar, él nos dará un `-1`, para este caso el cambio lo debemos hacer con el elemento mínimo de la lista, en primer lugar se obtiene el índice del elemento mínimo de la lista y posteriormente se hace el cambio.

```
1 else:
2     I_min = get_min_index(financial_reports)
3     financial_reports[I], financial_reports[I_min] =
        financial_reports[I_min], financial_reports[I]
```

3.4. Salida de datos

La salida debe imprimir el nombre del departamento acompañado de dos puntos, para esto se utiliza la variable `department`:

```
1 print(f"{department}:".)
```

Ahora, en la siguiente línea que se debe imprimir se debe imprimir los elementos de la lista separados por espacio, por esto mismo, se itera sobre los elementos de la lista de `financial_reports` y se modifica el parámetro de la función `print()` para que en lugar de imprimir un final de línea cada vez que se llame `print()` se imprima un caracter de espacio.

```
1 for i in financial_reports:
2     print(i, end=" ")
```

Por último, se imprime una nueva línea para que quede en el formato correcto para imprimir el siguiente departamento:

```
1 print()
```

3.5. Código final

Ensamblando cada parte del código obtenemos:

```
1 def get_max_index(lista: list[int]) -> int:
2     elemento_max = lista[0]
3     indice_max = 0
4     for i in range(len(lista)):
5         if (lista[i] > elemento_max):
6             elemento_max = lista[i]
7             indice_max = i
8     return indice_max
9
10 def get_min_index(lista: list[int]) -> int:
11     elemento_min = lista[0]
12     indice_min = 0
13     for i in range(len(lista)):
14         if (lista[i] < elemento_min):
15             elemento_min = lista[i]
16             indice_min = i
17     return indice_min
18
19 n = int(input())
20
21 while (n != 0):
22     for _ in range(n):
23         data = input().split()
24         department = data[0]
25         employees, I = int(data[1]), int(data[2])
26         financial_reports = list(map(int, input().split()))
27         I_max = get_max_index(financial_reports)
28
29         if (I != -1):
30             financial_reports[I], financial_reports[I_max] =
31                 financial_reports[I_max], financial_reports[I]
32         else:
33             I_min = get_min_index(financial_reports)
34             financial_reports[I], financial_reports[I_min] =
35                 financial_reports[I_min], financial_reports[I]
36
37         print(f"{department}:")
38         for i in financial_reports:
39             print(i, end=" ")
40         print()
41
42     n = int(input())
```



UTADEO

UNIVERSIDAD DE BOGOTÁ JORGE TADEO LOZANO